

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №1

по дисциплине «Управление разработкой промышленного программного обеспечения»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.т.н., доцент ОИТ

_____ Небаба С.Г.

Задание

Командная оболочка (она же – shell, или «шелл») – командный интерпретатор, используемый, в частности, в операционных системах Unix. На виртуальных машинах развернута ОС Linux Mint MATE. Linux – это семейство Unix-подобным ОС на базе ядра Linux. При выполнении практических работ на личной машине можно использовать любой дистрибутив Linux по вашему желанию.

Командная оболочка принимает от пользователя команды и переводит их в инструкции, понятные для ОС и также, при необходимости, возвращает выходные данные пользователю.

После того как Вы авторизовались в системе и открыли терминал, выполните следующие задания:

1. Получите информацию о командной оболочке с помощью команды `echo $SHELL`. Вы получили данные, которые понадобятся Вам для дальнейшего написания скрипта. Данная информация пригодится Вам позже.
2. Установите Python (python3). Вам понадобятся расширенные права доступа. Используйте команду `sudo`.
3. Найдите куда установлен Python (соответствующую команду Вам нужно найти самостоятельно).
4. Найдите имя и версию ОС (подсказка: поможет `cat` и обращение к каталогу `/etc`).
5. Реализуйте .sh-скрипт, который выполняет следующее:
 - Выполняется с использованием обязательного указания 1-й опции. Варианты опций: 3.
 - Опции могут быть любой буквы по Вашему выбору.
 - При использовании первой опции, пользователь получает вывод версии Python на экран.

- При использовании второй опции, пользователь получает вывод версии Linux.
- При использовании третьей опции пользователю выводится «пасхалка» Zen of Python.
- При использовании четвертой опции пользователю выводится информация о разработчике скрипта (достаточно указать свои ФИО).
- Без опций происходит ошибка и выход.

ПРИМЕЧАНИЕ: обратите внимание на пункт 1. В начале скрипта первая строка имеет вид `#!/<путь_к_интерпретатору>`, путь Вы получаете при выполнении пункта 1.

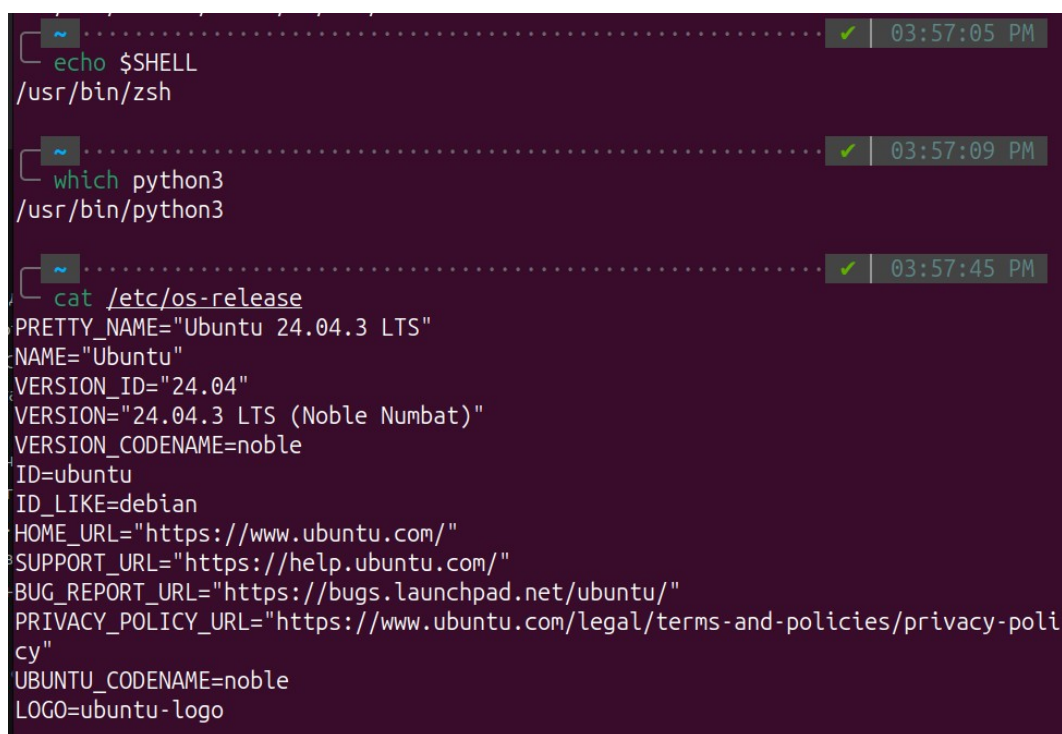
Подсказка: Выбор опции реализуется с помощью case и позиционных параметров.

Ход работы

В терминале была выполнена команда `echo $SHELL`. В результате получен путь к командной оболочке: `/usr/bin/zsh`. Данный путь используется в дальнейшем для указания интерпретатора в шебанг-строке скрипта.

С помощью команды `which python3` проверено наличие интерпретатора Python 3. Вывод `/usr/bin/python3` подтвердил, что Python установлен и дополнительная установка не потребовалась.

Для получения информации об операционной системе использована команда `cat /etc/os-release`. В выводе определены: `NAME="Ubuntu"`, `VERSION="24.04.3 LTS (Noble Numbat)"`. Эти данные предназначены для вывода версии Linux. Результаты выполнения команд представлены на рисунке 1.



```
~ ..... 03:57:05 PM
echo $SHELL
/usr/bin/zsh

~ ..... 03:57:09 PM
which python3
/usr/bin/python3

~ ..... 03:57:45 PM
cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.3 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

Рисунок 1. Команды заданий 1-4 и их результаты.

Затем был создан файл скрипта `script.sh` с помощью команды `touch`. В первой строке файла помещена шебанг-строка `#!/usr/bin/zsh`. В скрипте реализована проверка наличия аргумента: если `$#` равно 0, выводится сообщение об ошибке и инструкция по использованию. Обработка переданной

опции выполнена конструкцией case "\$1". Определены четыре допустимые опции:

- -p – вывод версии Python (python3 --version);
- -l – вывод названия и версии дистрибутива (grep -E "^(NAME|VERSION)=" /etc/os-release);
- -z – вывод пасхалки Zen of Python (python3 -c "import this");
- -a – вывод информации о разработчике (заранее заданные ФИО).

Ветка * обрабатывает недопустимые опции, выводя сообщение об ошибке и список допустимых опций. Исходный код скрипта представлен на рисунке 2.

```
1  #!/usr/bin/zsh
2
3  # Проверка наличия хотя бы одного аргумента
4  if [ $# -eq 0 ]; then
5      echo "Ошибка: опция не указана."
6      echo "Использование: $0 { -p | -l | -z | -a }"
7      exit 1
8  fi
9
10 # Обработка первого аргумента
11 case "$1" in
12     -p)
13         echo "Версия Python:"
14         python3 --version
15         ;;
16     -l)
17         echo "Версия Linux (дистрибутив):"
18         grep -E "^(NAME|VERSION)=" /etc/os-release
19         ;;
20     -z)
21         echo "Zen of Python:"
22         python3 -c "import this"
23         ;;
24     -a)
25         echo "Разработчик скрипта: Шаврин Алексей Павлович"
26         ;;
27     *)
28         echo "Ошибка: неверная опция '$1'."
29         echo "Допустимые опции: -p, -l, -z, -a"
30         exit 1
31         ;;
32 esac
33
34 exit 0
```

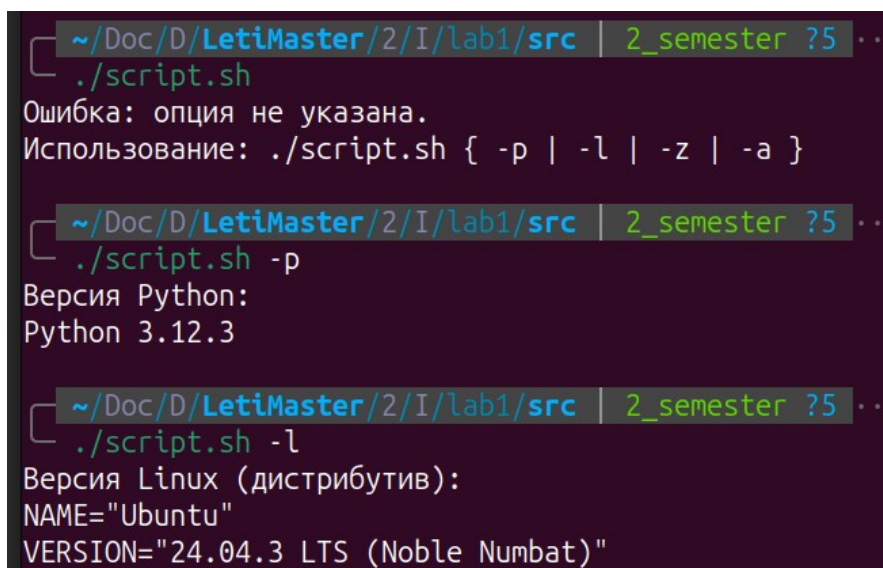
Рисунок 2. Разработанный скрипт.

После сохранения файла скрипту предоставлены права на выполнение командой `chmod +x script.sh`. Выполнены следующие проверки работы скрипта:

- `./script.sh -p` – выведена версия Python 3.12.3;

- `./script.sh -l` – выведены `NAME="Ubuntu"` и `VERSION="24.04.3 LTS (Noble Numbat)"`;
- `./script.sh -z` – выведен текст Zen of Python;
- `./script.sh -a` – выведены указанные ФИО;
- `./script.sh` (без аргументов) – выведено сообщение об ошибке и подсказка по использованию;
- `./script.sh -x` – выведено сообщение о неверной опции с перечнем допустимых.

Все тесты завершились без ошибок, скрипт отработал в соответствии с заданием. Результаты запуска скрипта с различными опциями представлены на рисунках 3-4.



```

~/Doc/D/LetiMaster/2/I/lab1/src | 2_semester ?5 ...
./script.sh
Ошибка: опция не указана.
Использование: ./script.sh { -p | -l | -z | -a }

~/Doc/D/LetiMaster/2/I/lab1/src | 2_semester ?5 ...
./script.sh -p
Версия Python:
Python 3.12.3

~/Doc/D/LetiMaster/2/I/lab1/src | 2_semester ?5 ...
./script.sh -l
Версия Linux (дистрибутив):
NAME="Ubuntu"
VERSION="24.04.3 LTS (Noble Numbat)"

```

Рисунок 3. Результаты выполнения скрипта с опциями p, l и без опций

```
~/Doc/D/LetiMaster/2/I/lab1/src | 2_semester ?5 ..... 1 x | 04:08:17 PM
./script.sh -z
Zen of Python:
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

~/Doc/D/LetiMaster/2/I/lab1/src | 2_semester ?5 ..... ✓ | 04:25:23 PM
./script.sh -a
Разработчик скрипта: Шаврин Алексей Павлович

~/Doc/D/LetiMaster/2/I/lab1/src | 2_semester ?5 ..... ✓ | 04:25:26 PM
./script.sh -h
Ошибка: неверная опция '-h'.
Допустимые опции: -p, -l, -z, -a
```

Рисунок 4. Результаты выполнения скрипта с опциями z, a и неопределенная опция h.

Вывод

В процессе выполнения работы получены навыки определения текущей командной оболочки, проверки наличия интерпретатора Python, извлечения информации об операционной системе из системных файлов. Освоено создание исполняемых shell-скриптов с обработкой аргументов командной строки с использованием условного оператора case, реализована проверка обязательности опции и обработка ошибочных ситуаций. Полученные результаты соответствуют поставленной задаче.